

UNIT STUDY NOTES

Production-Ready Python Mindset

Code that only works on the author's machine is not finished. This unit covers clean readable code, modular design, logging and monitoring habits, and how to read unfamiliar code deliberately.

 6 source notebooks

 31 topic sections extracted

 7 study sections

Unit Overview

 01 — Clean/Readable Code: The 6 Laws of Straight-Forward Style! 🍷

Topic 14.01 | *ProductionReadyMindset* — Part 1 of 4

 **Hook:** You wrote code that today's-genius understood; 3 MONTHS later you read it yourself and say "which MADMAN wrote this?!" 😂 Code is not just for the compiler — it's for HUMANS! In interviews/production/quick-fixes the most VALUABLE skill = READABLE code!

The 6 Laws (rules-of-thumb!):

1. **Names = the story** — say WHOSE, WHAT (a, x1 ❌ → `student_avg` ✅)
2. **Small functions** — one function, one answer (a ~20-line ceiling!)
3. **DRY** — copy-paste = the clone-army of future bugs 🐛
4. **Magic-numbers = ghosts** — 0.9?? put it in constants!
5. **Comments = WHY, code = WHAT** — `i += 1 # i badhata` 😞 worthless!
6. **Dead-code = delete** — git-history keeps the memory — scrap it! 🗑️

 **WHY:** bug-hunting 10x faster, adding features 3x faster, and the team loves you! ❤️

  02 — Before/After: 2 Makeover Cases (UGLY → CLEAN)

Topic 14.01 | *ProductionReadyMindset* — Part 1 of 4 (examples notebook)

 **Hook:** Not a beauty-parlour, a CODE-parlour 🧑: before = "whatever-just-works- somehow code", after = "an OPA readable version". Same answers — SAME outputs — but the reader's eyes will say THANKS!

Makeover-1: sensor-spam log lines 📄 · Makeover-2: the warehouse bill-goblin 📄

 01 — Modular Design: Different Master-Craftsmen for Each Room of the House! 🏠

Topic 14.02 | *ProductionReadyMindset* — Part 2 of 4

 **Hook:** You built a 1000-line single-slab — one giant script holding CSV too, validation too, plotting too, billing too! Then the ELECTRICIAN's job came up → you had to demolish the whole house!! 🧑 Pro-builders don't ship a single piece: kitchen-team, electric-team, plumber-team — each one builds its own ROOM! That's

what we call in code: **MODULE** (one .py = one craftsman-team!) 🧩

📖 Definitions:

- **module** = one `.py` file — its own functions/constants
- **package** = a folder + `__init__.py` (the team's tower!)
- **import-scene** = use another team's work: `import team_stats` 🔗
- `if __name__ == "__main__":` = demo when run AS a script; SILENT when imported — the guard-master! (no side-effects on module load) 🛡️

🔍 **WHY modular?** smaller files → testable → zero team-collisions → reuse (marks_stats will work in every project!)

🏗️ 02 — Refactoring Demo: MEGA-Script → 3-Module Tower!

Topic 14.02 | *ProductionReadyMindset* — Part 2 of 4 (demo notebook)

🌐 **Hook:** A sensor-ETL script — a 100-line THEPLA: reading, cleaning, avg-spotting, report-printing all in one file. Problem: the cramped work — you go to fix the parser and the REPORT breaks! 🤖 From today on it's live: the MEGA's demolition, 3 small rooms (load → compute → report) — let the tower stand! 🏗️

🔧 01 — Logging & Monitoring Mindset: The Black-Box Mentality!

Topic 14.03 | *ProductionReadyMindset* — Part 3 of 4

🌐 **Hook:** A flight crashes — the first investigation looks at the BLACK-BOX: "what did the plane SAY in its last 1 hour?!" On a production-server too, your app-crash is the same thing: the boss asks "when did it come, what happened?" — only logs can tell! `print` = writing a note on paper and throwing it in the wind 📄 🗑️ — a logger = the black-box!

📖 MINDSET-First (you learned the syntax in §11, now the THINKING):

THOUGHT	DIRECT MEANING
WHICH level?	DEBUG=developer-diary · INFO=work-flowing 🟢 · WARNING=heavy-breathing! 🟡 · ERROR=fell! 🔴 · CRITICAL=fire-alarm! 🔥
WHAT to log?	events + decisions + numbers (rows→parse→avg!); clues for future fixes
WHAT NOT to log?	PASSWORDS/tokens/cards — always mask/no-store ⚠️
Monitoring = health-checks + dashboards ON TOP of logs (the alert-pyramid!)	

🔍 **WHY the mindset?** library-level syntax takes 1 hour to learn; the real-born-thinking of logging (what, how much, when to step up) — that's what makes 10x engineers!

📖 🧑 01 — Reading Others' Code: The GPS for New Terrain!

Topic 14.04 | *ProductionReadyMindset* — Part 4 of 4 (finale!)

🌐 **Hook:** Your first day on the job: the team says "this repo is yours!" — 500 files... WHAT?! 🤖 Pro-devs don't read code first, they draw a MAP first: where's the entry point? What are the call-chains in order? → other people stay confused for 2 hours → you have a map in 20 minutes! This skill gets the red-hot talent-need met ✨

📖 The 5-Step Navigator Strategy (page-turning-friendship!):

1. **README/entry FIRST** — where does the story begin? (`main.py/cli.py/README` !)
2. **TREE-VIEW** — folder names reveal the grouping of things (io/ viz/ calc/?)
3. **Trace ONE REQUEST** — one full journey (entry → helpers → return!)
4. **Tests = living-docs** 📄 — read the tests to understand the API's real-behavior!
5. **SMALL changes + verify** — the first edit is a small rehearsal 🎭

In this notebook we build a mini **mystery-repo** (3 files!) — and apply that very strategy to it LIVE! 🕵️

Prerequisites

→ Modules, Packages and Environments

What you will be able to do

- ✓ Write code that a reviewer can follow without you in the room
- ✓ Structure projects into modules with clear boundaries
- ✓ Instrument code with logging instead of print debugging
- ✓ Approach an unfamiliar codebase systematically

What This Unit Covers

CONCEPT MAP

readability and naming



modularity and project structure



logging, monitoring and observability



reading other people's code

The sections below follow this arc, from foundational semantics to the engineering decisions that follow from them.

Core Concepts

01cleancodeprinciplestheory

Level 0 — The Name's Story-Test 🧑

PYTHON

```
# ❌ GANDA (padhke bhi matlab zero):
def calc(l):
    t = 0
    for v in l:
        t += v * 0.9          # 0.9?? fest-discount?! 🤖 mystery-ghost!
    return t / len(l)

# ✅ CLEAN (seedha-paani, kahani-bolta!):
DISCOUNT_RATE = 0.9        # shop-wide festive-discount 🇮🇳 (ghost = CAPTURED!)

def average_discounted_price(prices: list[float]) -> float:
    """har price pe discount laga ke average 📄"""
    return sum(p * DISCOUNT_RATE for p in prices) / len(prices)

prices = [100, 200, 350]
print("ganda-answer :", calc(prices))
print("clean-answer :", average_discounted_price(prices), "(same math, NAYI story!)")
# Expected OUTPUT:
# ganda-answer : 195.0
# clean-answer : 195.0 (same math, NAYI story!)
```

Level 1 — Small Functions = Small Blame! 🔨

Getting "load+parse+filter+report" all inside one function = chaos-storm! Break it apart:

Functional tools

map/filter apply functions; reduce aggregates. Often paired with lambdas.

PYTHON

```
# ❌ MEGA-FN (1 darwaza, 5 kaam):
def mega(filepath):
    raw = open(filepath).read()
    rows = []
    for line in raw.splitlines():
        if line.startswith("#") or not line.strip():
            continue
        name, mark = line.split(":")
        if int(mark) < 0:
            continue
        rows.append((name, int(mark)))
    avg = sum(m for _, m in rows) / len(rows)
    return f"students={len(rows)} avg={avg:.1f}"

# ✅ 3 CHHOTE (har-ek ek-jawaab, har-ek TESTABLE!):
def load_marks_file(filepath: str) -> list[tuple[str, int]]:
    """file -> (name, mark) rows - comments/blank/negative OUT!"""
    rows = []
    for line in open(filepath):
        if line.startswith("#") or not line.strip():
            continue
        name, mark = line.split(":")
        if int(mark) >= 0:
            rows.append((name.strip(), int(mark)))
    return rows

def class_average(rows: list[tuple[str, int]]) -> float:
    return round(sum(m for _, m in rows) / len(rows), 1) if rows else 0.0
# ...

print("chhotte-se-bani report:", class_report(rows))
# Expected OUTPUT:
# rows-load: [('Asha', 95), ('BK', 75), ('Chirag', 100), ('Diya', 77)]
# chhotte-se-bani report: students=4 avg=86.8
```

5 Classic Mistakes (Clean-Code)

#	❌ MISTAKE	✅ CORRECT
1	<code>calc/process/handle</code> zero-meaning names	verb+noun: <code>average_discounted_price</code>
2	comments ECHO (worthless <code>i+=1 # +1</code>)	write WHY, let code say WHAT
3	copy-paste the same-logic everywhere	make a fn, stay DRY 📄
4	<code>if status == 3:</code> (3???)	constants: <code>PAID = 3</code> 🍌
5	keeping dead-code ON-hold (future-plans!)	git-history remembers — DELETE! 🗑️

🚀 Recap — 30 Seconds

- Story-speak naming 🏰 · small fns ✅ · DRY 🐛 · magic→constants 🍌

- Comments = WHY · Dead-code = remove it (git will remember!)

Next: `02_before_after_examples.ipynb` — real-feel MESSY→CLEAN makeovers (the expose of sensor-spam & warehouse-bill!) 🌸

02beforeafter_examples

Makeover-1: Sensor-Scam Separator 🚩 (BEFORE)

Loops

for iterates over iterables; while runs while condition true. Use break/continue/else appropriately.

When to use

Repetitive tasks, processing collections.

PYTHON

```
# ❌ BEFORE – anonymous-field guessing game (L[1]? L[3]?? kon? kya?!)
def old_parse(lines):
    out = []
    for l in lines:
        p = l.split("|")
        if len(p) >= 3 and p[2] == "OK":
            out.append((p[0], float(p[1])))
    tot = 0
    for _, v in out:
        tot += v
    return out, round(tot / len(out), 1) if out else 0

RAW = ["s1|42.5|OK", "s2|17.3|ERR", "s3|60.0|OK", "note|x|OK?"]
res, avg = old_parse(RAW)
print("before-out:", res, "| avg:", avg)
# Expected OUTPUT:
# before-out: [('s1', 42.5), ('s3', 60.0)] | avg: 51.2
```

Makeover-1: AFTER (names! constants! docstrings! small fns!)

PYTHON

```

STATUS_OK = "OK" # kabhi "OKAY"/"ok" nahi — ek-hi shabd! 🎯

def parse_sensor_line(line: str):
    """s1/42.5/OK → ("s1", 42.5) | wahi-line status OK nahi → None"""
    parts = line.split("/")
    if len(parts) < 3 or parts[2] != STATUS_OK:
        return None
    return parts[0], float(parts[1])

def average_of(keepers: list) -> float:
    return round(sum(v for _, v in keepers) / len(keepers), 1) if keepers else 0.0

def sensor_sift(lines: list[str]):
    keepers = [k for k in (parse_sensor_line(l) for l in lines) if k]
    return keepers, average_of(keepers)

res2, avg2 = sensor_sift(RAW)
print("after-out :", res2, "| avg:", avg2)
assert (res, avg) == (res2, avg2) # SAME outputs, NAYI readability! 🎯
print("✅ same-answer proof — parlour seva success!")
# Expected OUTPUT:
# after-out : [('s1', 42.5), ('s3', 60.0)] | avg: 51.2
# ✅ same-answer proof — parlour seva success!

```

3 Makeover-Specific Mistakes

#	 MISTAKE	 CORRECT
1	changing the BEHAVIOR-TOO during cleanup	FIRST pin the same-output assert! 🎯
2	refactoring everything into one fn (just renamed)	split into one-task-one-fn
3	decoration over depth — only the looks	constants + docstrings + guard-returns

Recap — 15 Seconds

- SAME-output proof = the makeover's passport  (assert old==new!)
- line-parser / average / bill-rules = testable-tiny-pieces

Next topic: [02_Modular_Design](#) — 1 big-script → 3 clean MODULES (learn the import-scene, LIVE with files!)



01modulardesign_theory

Level 0 — The `__name__` Mirror 🪞 (the guard-master LIVE!)

PYTHON

```
# notebook==script-direct - __name__ hamesha "__main__"!
print("yeh file chal rahi as:", __name__)

# module jagah pe dikhein: report ka madhna daan sirf direct-chalane pe - guard:
def demo_only():
    return "main-guard ke ANDAR wala kaam (import pe SILENT!) 🧠"

if __name__ == "__main__":
    print("guard-inside:", demo_only()) # import se chala toh SKIP!
# Expected OUTPUT:
# yeh file chal rahi as: __main__
# guard-inside: main-guard ke ANDAR wala kaam (import pe SILENT!) 🧠
```

Level 1 — Your Own Module: Write + Import (LIVE!) 📦

Let's make a file `marktools_module.py` (real file work!): load/clean fns — then import it and RUN it — module-mechanics LIVE! (plus cleanup!)

PYTHON

```

from pathlib import Path
import importlib, sys

# 1) module likho (asali-file, permanent Lalit-pur pattern!)
Path("marktools_module.py").write_text('''
"""marktools – parse/format helpers (demo-module!) 🌱 """

def parse_mark_line(line: str):
    """Asha: 95" → ("Asha", 95); kharab → None"""
    if ":" not in line:
        return None
    name, mark = line.split(":")
    return name.strip(), int(mark)

def pretty_rows(rows) -> str:
    return ", ".join(f"{n}={m}" for n, m in rows)

if __name__ == "__main__":
    print("SELF-DEMO:", pretty_rows([parse_mark_line("Asha: 95")]))
''')

# 2) import karo (cwd path pe hai – sys.path me "." hote hi!)
sys.path.insert(0, ".")
importlib.invalidate_caches() # fresh-written files ETIC-recognize ete!
mt = importlib.import_module("marktools_module")

# 3) chalao!
r1 = mt.parse_mark_line("Asha: 95")
# ...

# Expected OUTPUT:
# parse-good: ('Asha', 95) | parse-bad: None
# pretty      : Asha=95, BK=75
# cleanup ✓ (module kaam kar gaya, files saaf!)

```

Level 2 — Package-Tower + the Import-Cycle Trap 🚧 🔗

Package = a folder + `__init__.py`: several door-keeper modules of one team. **CYCLE-TRAP:** `a` imports `b`, `b` imports `a` → an import-lock! 🚫 **CURE:** move the shared thing into a THIRD module (c-team!) — a linear flow!

PYTHON

```

Path("shoptower").mkdir(exist_ok=True)
Path("shoptower/__init__.py").write_text('"""shoptower package – godown-team!"""' + '\nfrom .pricing import apply_discount')
Path("shoptower/pricing.py").write_text('
def apply_discount(total: float, rate: float = 0.9) -> float:
    return round(total * rate, 2)

if __name__ == "__main__":
    print("SELF:", apply_discount(100))
')

import shoptower.pricing as spr
from shoptower import apply_discount # __init__ ne EXPOSE kiya! ✨
print("dot-import :", spr.apply_discount(200))
print("pkg-expose :", apply_discount(200), "(tallest-ka-direct!)")

shutil.rmtree("shoptower", ignore_errors=True); shutil.rmtree("__pycache__", ignore_errors=True)
print("tower demolish ✓ prototype-teach kaam complete!")
# Expected OUTPUT:
# dot-import : 180.0
# pkg-expose : 180.0 (tallest-ka-direct!)
# tower demolish ✓ prototype-teach kaam complete!

```

5 Classic Mistakes (Modular)

#	✗ MISTAKE	✓ CORRECT
1	import-time side-effects (code runs!)	<code>__main__</code> -guard 🛡️
2	import-cycle (a↔b deadlock!) 🚫	shared thing → third-module (c-team!)
3	importing everything via <code>from x import *</code>	explicit-select (clean namespace!)
4	mixing teams in one file (CSV+plot+billing)	one-team-one-file 🧩
5	hardcoding absolute-paths <code>C:\data\...</code>	relative-paths/config (portable!)

Recap — 30 Seconds

- module = .py-team 📦 · package = folder + `__init__.py` 🏠
- `__name__ == "__main__"` = the demo-guard (SILENT on import!)
- the cure for a cycle = a third-party-module; `__init__` gives the exposure-polish ✨

Next: `02_refactoring_demo.ipynb` — the complete sectioning-by-tearing of 1 MEGA-script: a 3-module-tower built LIVE (load→compute→report!) 🏗️

02refactoringdemo

BEFORE — MEGA `sensor_etl.py` (concept-print!)

Anyone touching this = EVERYONE confused. We'll split it into `etl_load` / `etl_compute` / `etl_report` !

PYTHON

```
# 1-file-etl (meiodule-style ganda):
raw = open("sensors.csv").readlines()
rows = []
for l in raw:
    if l.startswith("#") or not l.strip(): continue
    p = l.split(",")
    if p[2].strip() == "OK":
        rows.append((p[0], float(p[1])))
tot = 0
for _, v in rows: tot += v
avg = round(tot / len(rows), 1)
print(f"rows={len(rows)} avg={avg}")
best = rows[0]
for r in rows:
    if r[1] > best[1]: best = r
print("top:", best[0], best[1])
# + report lines CSV Likhna + email-format - SAB YAHIN! 🙋
```

Step 1 — Demo-CSV Seed 🛒 (these are the supplies!)

PYTHON

```
from pathlib import Path

Path("sensors_demo.csv").write_text(
    "# log-start\ns1,42.5,OK\ns2,17.3,ERR\ns3,60.0,OK\ns4,55.2,OK\ns5,8.0,OK\n")
print(Path("sensors_demo.csv").read_text())
# Expected OUTPUT:
# # log-start
# s1,42.5,OK
# s2,17.3,ERR
# s3,60.0,OK
# s4,55.2,OK
# s5,8.0,OK
```

Step 2 — Write the 3 Modules (each-room-expert!) 🌱

PYTHON

```
# ROOM 1 – etl_load.py: sirf file-parsing (csv→rows!)
Path("etl_load.py").write_text(''
'''etl_load – CSV-file ko saaf rows deta room 📄'''

def load_ok_rows(fp: str) -> list[tuple[str, float]]:
    rows = []
    for line in open(fp):
        if line.startswith("#") or not line.strip():
            continue
        name, val, status = (x.strip() for x in line.split(","))
        if status == "OK":
            rows.append((name, float(val)))
    return rows

if __name__ == "__main__":
    print("SELF:", load_ok_rows("sensors_demo.csv"))
    ''')

# ROOM 2 – etl_compute.py: sirf maths (avg/top!)
Path("etl_compute.py").write_text(''
'''etl_compute – rows ko stats 📊'''

def rows_avg(rows) -> float:
    return round(sum(v for _, v in rows) / len(rows), 1) if rows else 0.0

def rows_top(rows):
    return max(rows, key=lambda r: r[1]) if rows else None

# ...

print("3 rooms likh diye ✓ (etl_load 📄 / etl_compute 📊 / etl_report 🖨️)")
# Expected OUTPUT:
# 3 rooms likh diye ✓ (etl_load 📄 / etl_compute 📊 / etl_report 🖨️)
```

Step 3 — MAIN Runner: Tower Built! 🏰 (import → chain → cleanup)

PYTHON

```
import sys, importlib, shutil
sys.path.insert(0, ".")

importlib.invalidate_caches() # fresh-written files ETIC-recognize etc!
etl_load = importlib.import_module("etl_load")
etl_compute = importlib.import_module("etl_compute")
etl_report = importlib.import_module("etl_report")

rows = etl_load.load_ok_rows("sensors_demo.csv")
avg = etl_compute.rows_avg(rows)
top = etl_compute.rows_top(rows)
report = etl_report.make_report(rows, avg, top)
print(report)

assert rows == [("s1", 42.5), ("s3", 60.0), ("s4", 55.2), ("s5", 8.0)], "4 OK rows!"
assert avg == 41.4 and top == ("s3", 60.0), "avg 41.43→41.4, top=s3@60!"

# cleanup - demo-artifacts asali-repo me rakhte, NB me saaf!
for f in ("etl_load.py", "etl_compute.py", "etl_report.py", "sensors_demo.csv"):
    Path(f).unlink()
shutil.rmtree("__pycache__", ignore_errors=True)
print("tower tour ✓ - files saaf (asali repo me YEH structure permanents & survives!) 🏰")
# Expected OUTPUT:
# SENSOR-CHAAT
# rows=4 avg=41.4
# champion=s3 @ 60.0
# credentials: load→compute→report tower! 🏰
# tower tour ✓ - files saaf (asali repo me YEH structure permanents & survives!) 🏰
```

3 Refactor-Specific Mistakes

#	❌ MISTAKE	✅ CORRECT
1	DIRECT-run logic lying open in module files	every file: a <code>__main__</code> -guard 🛡️
2	chain = import-cycle (load→report→load!) 🔒	one-way flow (load→compute→report!)
3	skipping tests during the refactor (no same-output proof!)	asserts on top/avg EXACT

🚀 Recap — 15 Seconds

- The 3-room pattern: 📁 load (files) → 🧮 compute (pure!) → 🖨️ report (display)
- the reward of modularity: fix the parser, and the report doesn't break! 💪

Next topic: `03_Logging_Monitoring_Mindset` — the "what to log, WHERE to log, HOW to drink it" mindset (the BLACK-BOX flight-recorder of production!) 🚀

01loggingmindset_theory

Level 0 — Black-Box Setup Live: Timestamps + Levels + FILE 📦

(the syntax §11-05 buddy-pair — now with a standard-shipping vibe: time-name-level!)

PYTHON

```
import logging
from logging.handlers import RotatingFileHandler

log = logging.getLogger("chaihut-etl 🦄")
log.setLevel(logging.DEBUG)
for h in log.handlers[:]: log.removeHandler(h) # rebuild-safety!

handler = RotatingFileHandler("blackbox_demo.log", maxBytes=220, backupCount=2)
handler.setFormatter(logging.Formatter("%(asctime)s | %(levelname)s | %(name)s | %(message)s"))
log.addHandler(handler)

log.info("ETL shuru 🟢")
log.debug("parse-attempt batch-1 (dev-detail!)")
log.warning("s2 line me ERR status o strange? 🟡")
log.error("s9 parse FAIL - column-count mismatch 🔴")
log.critical("avg=0 aaya - poori pipeline rok di! 🚨")
log.info("config-file loaded ✓")
log.info("rows=417 processed-sync")
log.info("ETL khatam 🏁")

print("blackbox lines:", open("blackbox_demo.log").read())
# Expected OUTPUT (style - timestamps tumhare run-wale!):
# YYYY-MM-DD hh:mm:ss | INFO | chaihut-etl 🦄 | ETL shuru 🟢
# ...
# (8+ formatted lines dikhni chahiye - STATIC-style, level-wise emoji-Labels!)
```

Level 2 — The Monitoring-Pyramid: ON TOP of Logs (concepts!) 🚨

- **Heartbeat** = an info-line every 15-min: "I am alive ❤️" (silence = a death-signal!)
- **Count-metric** = counts mined by parsing logs ("parse-fail% > 2" → alert!)
- **DS-pipelines**: compare rows-in vs rows-out — hidden LOSSES detect data-bugs!

PYTHON

```
🚨 ALERTS (page/mail: "ERROR>10/min chal rahi!" 📧 HUMAN jogao📧)
📊 DASHBOARDS (metrics: rows/min, error-rate, latency 📊 NUMBERS-ka-satellite📊)
📦 LOGS (har-event ka raw-dera 📦 black-box📦)
```

5 Classic Mistakes (Logging-Mindset)

#	❌ MISTAKE	✅ CORRECT
1	<code>print</code> in production (vanishes in the air)	logger + handlers (file/rotation!)
2	directly logging PASSWORD/tokens POST-payload ⚠️	mask/ids-only, audit-friendly!
3	making EVERYTHING DEBUG-level (unreadable soup!)	mix the levels: diary/context/alarm-order
4	one file growing to GBs (open-crash!)	RotatingFile(maxBytes, backupCount) 🌀
5	write the logs but never look at them	heartbeat+alerts (monitoring ON TOP of logs!)

🚦 Recap — 30 Seconds

- The mindset = which-level / what-to-log / what-not-to-log (privacy-first!) 🛡️
- Black-box formatter: time+level+name+msg ✂️ · Rotation = the overflow-magic 🌀
- Logs → metrics → dashboards → ALERTS = the monitoring-pyramid 📶

Next topic: `04_Reading_Others_Code` — a GPS-navigator for SOMEONE-ELSE's repo (entry→trace→test reading strategies, LIVE mystery-repo!) 📖👤

01readingcodestrategiestheory

Step 0 — Materialize a Mystery-Repo 📁 (the teaching spot!)

PYTHON

```

from pathlib import Path
import shutil

repo = Path("mystery_repo"); repo.mkdir(exist_ok=True)
(repo / "README.md").write_text("# Myst-StackBot v1\nLooks like a small CLI for marks-bands!\n")
(repo / "cli.py").write_text('''from core import band_report

def main():
    rows = [("Asha", 95), ("BK", 62), ("CD", 41)]
    print(band_report(rows))

if __name__ == "__main__":
    main()
''')
(repo / "core.py").write_text('''from utils import classify

def band_report(rows):
    bags = {"A": [], "B": [], "C": [], "F": []}
    for name, mark in rows:
        band = classify(mark)
        bags[band].append(name)
    lines = [f"{b}Band: {' '.join(bags[b]) or '-'}" for b in "ABCF"]
    return "\n".join(lines)
''')
(repo / "utils.py").write_text('''def classify(m):
    if m >= 85: return "A"
    if m >= 50: return "B"
    if m >= 35: return "C"
    # ...
# README.md (52b)
# cli.py (168b)
# core.py (290b)
# utils.py (120b)

```

Step 1+2 — README-Glimpse + TREE + ENTRY-Find 🔍

How do you find the entry point? LINE-GREP for the `__main__`-guard (the craftsman's pattern!)

Exceptions

Handle errors gracefully with try/except/else/finally. Avoid bare except; catch specific exceptions.

PYTHON

```

print("README peeks:", (repo / "README.md").read_text().splitlines()[1])

def find_entry_guard(root: Path):
    hits = []
    for py in sorted(root.glob("*.py")):
        for i, line in enumerate(py.read_text().splitlines(), 1):
            if '__name__' in line:
                hits.append(f"{py.name}:{i}: {line.strip()}")
    return hits

print("\n__main__-guards scan:", find_entry_guard(repo))
print("→ ENTRY = cli.py (fit line 6-8 guard!) 🎯")
# Expected OUTPUT:
# README peeks: Looks like a small CLI for marks-bands!
#
# __main__-guards scan: ['cli.py:6: if __name__ == "__main__":']
# → ENTRY = cli.py (fit line 6-8 guard!) 🎯

```

Step 3 — One-REQUEST Trace + Run (draw the map, verify it!) 🗺️

Functional tools

map/filter apply functions; reduce aggregates. Often paired with lambdas.

PYTHON

```
# Imports web: cli → core → utils (grep 'from/import' AND RUN-finally!)
chain = []
for py in ["cli.py", "core.py", "utils.py"]:
    first = (repo / py).read_text().splitlines()[0]
    chain.append(f"{py}: {first}")
print("import-chain scan:")
for c in chain: print("  ", c)

import importlib.util, sys
sys.path.insert(0, "mystery_repo")
importlib.invalidate_caches()
import cli
print("\n--- execute entry (entry=run-path verified!) ---")
rows = [("Asha", 95), ("BK", 62), ("CD", 41)]
from core import band_report
print(band_report(rows))

# now map done → clean up (demo repo tum kaam-parciyal-ghosts-chhodenge!)
shutil.rmtree("mystery_repo", ignore_errors=True)
shutil.rmtree("__pycache__", ignore_errors=True); sys.path.remove("mystery_repo")
print("\nmap-complete ✓ mystery solved → cleaned! 🧹")
# Expected OUTPUT:
# import-chain scan:
#   cli.py: from core import band_report
#   core.py: from utils import classify
#   utils.py: def classify(m):
#
# --- execute entry (entry=run-path verified!) ---
# ABand: Asha
# BBand: BK
# CBand: CD
# FBand: -
#
# map-complete ✓ mystery solved → cleaned! 🧹
```

5 Classic Mistakes (Reading-Code)

#	❌ MISTAKE	✅ CORRECT
1	STARTING at a random-file (directionless!)	entry-scan + tree first 📖
2	wanting to see EVERYTHING at once (brain-burn!)	trace ONE request, gradually
3	blindly trusting the comments	behavior = the code (comments can lie!)
4	skipping the tests while reading	tests = the API's living-reference 🟢
5	first-change MASTER-massive	a small-rehearsal edit + run-verify 🔄

🔴 Recap — 30 Seconds

- 5 steps: README-entry → TREE → ONE-trace → TESTS-docs → small-edits 📌
- **main-grep** = the entry-locator 🎯 · import-chain scan = the call-map 📖
- old-map and new-map pull the same weight — close it with a runnable-verify!

🎉 **Section 14 = mindset-set complete!** CLEAN → MODULAR → BLACK-BOX → READER — the production-engineer's INNER-OS installed! 📦

Edge Cases and Boundaries

LEVEL 1 — THE MAGIC OF ROTATION: OVERFLOW = SELF-ARCHIVE! 🌀

Once the file fills past maxBytes=220 → `backupCount=2` auto-archives it! In production this fights the file-growing-to-death problem! Watch the overflow live:

Real-World and AI Engineering Relevance

Where this shows up in AI engineering

- readability and naming
- modularity and project structure
- logging, monitoring and observability
- reading other people's code

EXAM AUR INTERVIEW PERSPECTIVE

Agar koi concept interview ya exam mein poochha jaye, to definition ke saath apna ek chhota practical example zaroor bolo. Sirf theory batane se confidence nahi dikhta - implementation dikhane se dikhta hai.

Quick Reference

Short, representative snippets collected from the unit's notebooks. Use them as syntax reminders, not as complete programs.

PYTHON

LEVEL 0 — THE `\_\_NAME\_\_` MIRROR 🪞 (THE GUARD-MASTER LIVE!)

```
# notebook==script-direct - __name__ hamesha "__main__"!
print("yeh file chal rahi as:", __name__)

# module jagah pe dikhein: report ka madhna daan sirf direct-chalane pe - guard:
def demo_only():
    return "main-guard ke ANDAR wala kaam (import pe SILENT!) 🤫"

if __name__ == "__main__":
    print("guard-inside:", demo_only()) # import se chala toh SKIP!
# Expected OUTPUT:
# yeh file chal rahi as: __main__
# guard-inside: main-guard ke ANDAR wala kaam (import pe SILENT!) 🤫
```

PYTHON

STEP 1 — DEMO-CSV SEED 🛒 (THESE ARE THE SUPPLIES!)

```
from pathlib import Path

Path("sensors_demo.csv").write_text(
    "# log-start\ns1,42.5,OK\ns2,17.3,ERR\ns3,60.0,OK\ns4,55.2,OK\ns5,8.0,OK\n")
print(Path("sensors_demo.csv").read_text())
# Expected OUTPUT:
# # log-start
# s1,42.5,OK
# s2,17.3,ERR
# s3,60.0,OK
# s4,55.2,OK
# s5,8.0,OK
```

PYTHON

LEVEL 2 — THE MONITORING-PYRAMID: ON TOP OF LOGS (CONCEPTS!) ⚠️

```
🚨 ALERTS (page/mail: "ERROR>10/min chal rahi!" 🚫 HUMAN jogao🚫)
🚫
📊 DASHBOARDS (metrics: rows/min, error-rate, latency 🚫 NUMBERS-ka-satellite🚫)
🚫
📦 LOGS (har-event ka raw-dera 🚫 black-box🚫)
```

Revision Checklist

Work through this list without looking at the notes. Anything you cannot explain aloud belongs back in the revision queue.

☐ Level 0 — The Name's Story-Test 🧑🏻

- ☐ Level 1 — Small Functions = Small Blame! 🛠️
- ☐ 5 Classic Mistakes (Clean-Code)
- ☐ Makeover-1: Sensor-Scam Separator 🎨 (BEFORE)
- ☐ Makeover-1: AFTER ✅ (names! constants! docstrings! small fns!)
- ☐ 3 Makeover-Specific Mistakes
- ☐ Level 0 — The `__name__` Mirror 🪞 (the guard-master LIVE!)
- ☐ Level 1 — Your Own Module: Write + Import (LIVE!) 📦
- ☐ Level 2 — Package-Tower + the Import-Cycle Trap 🚧 🌀
- ☐ 5 Classic Mistakes (Modular)
- ☐ Recap — 30 Seconds
- ☐ BEFORE — MEGA `sensor_et1.py` (concept-print!)
- ☐ Step 1 — Demo-CSV Seed 🛒 (these are the supplies!)
- ☐ Step 2 — Write the 3 Modules (each-room-expert!) 🧩
- ☐ Step 3 — MAIN Runner: Tower Built! 🏰 (import → chain → cleanup)
- ☐ 3 Refactor-Specific Mistakes
- ☐ Level 0 — Black-Box Setup Live: Timestamps + Levels + FILE 📦
- ☐ Level 2 — The Monitoring-Pyramid: ON TOP of Logs (concepts!) 🚧
- ☐ 5 Classic Mistakes (Logging-Mindset)
- ☐ Step 0 — Materialize a Mystery-Repo 📁 (the teaching spot!)
- ☐ Step 1+2 — README-Glimpse + TREE + ENTRY-Find 🔍

☐ Step 3 — One-REQUEST Trace + Run (draw the map, verify it!) 📖